CHAPTER 18

Extended and Adaptive Thinking

In this chapter

Introduction: The Migration That Passed Every Test . . . 3
18.1 Two Modes, and Why One Is Legacy . . . 4
18.2 Turning It Off, Where You Can . . . 5
18.3 The Thinking Block . . . 5
18.4 display: Summarised, Omitted, and What Changes . . . 6
18.5 Effort Is Not Thinking . . . 7
18.6 Task Budgets . . . 7
18.7 Thinking with Tool Use: A Loop Is One Turn . . . 8
18.8 Preserving Thinking Blocks . . . 8
18.9 Redacted Thinking . . . 9
18.10 Interleaved Thinking . . . 9
18.11 Preservation by Model, and the Context Bill . . . 10
18.12 Thinking and Prompt Caching . . . 11
18.13 Thinking and the Context Window . . . 11
18.14 Limits and Feature Compatibility . . . 12
18.15 Visual Analogies . . . 12
18.16 Architecture: One Thinking Policy, Not Fifty Call Sites . . . 13
18.17 Examples: Four Configurations . . . 14
18.18 Code Walkthrough: A Preflight That Refuses to Guess . . . 15
18.19 Mini Project: The Thinking Preflight and the Context Accountant . . . 16
18.20 Panel A — The Matrix, and Three Cells That Refuse to Answer . . . 17
18.21 Panel B — Fourteen Documented Rejections . . . 18
18.22 Panel C — Nine Requests a Team Would Plausibly Write . . . 18
18.23 Panel D — The One-Word Bug . . . 19
18.24 Panel E — What a Model Switch Leaves Behind . . . 19
18.25 Panel F — The Context Accountant . . . 20
18.26 Panel G — The Configuration Is Part of the Prompt . . . 21
18.27 Evaluation and Deployment . . . 21
18.28 Hands-on Labs . . . 22

18.29 Lab Solutions 22
18.30 Best Practices 23
18.31 Common Mistakes 23
18.32 Performance Considerations 25
18.33 Security Considerations 25
18.34 Real Production Examples 25
18.35 Certification Notes 26
18.36 Twenty Practice Questions 27
18.37 Answers and Explanations 27
18.38 Ten Practical Exercises 28
18.39 Five Challenge Labs 29
18.40 Ten Interview Questions 29
18.41 Summary 29
18.42 Glossary 31
18.43 Further Reading 32

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Distinguish **adaptive thinking** from **legacy extended thinking**, say which models accept which, and configure both correctly.
2. Read a **thinking block** — `thinking`, `signature`, and the `redacted_thinking` variant — and explain what each field is for.
3. Choose a `display` setting knowing exactly what it changes: **latency, not cost**.
4. Separate `thinking` from `effort`, and pick the right one for a stated goal.
5. Apply the **preservation protocol**: pass thinking blocks back complete and unmodified, and know the one documented exception.
6. Predict which of your requests will return a **400** before sending them, from the per-model matrix.
7. Account for thinking in the **context window**, distinguishing keep-all models from keep-last-turn models and quantifying the difference.
8. Explain why changing the thinking configuration or the effort level **invalidates prompt caching**, and price the mistake.

NOTE

Prerequisites

Chapter 13 for what test-time compute buys and what it does not — `accuracy ≈ coverage × selection` — and for the thinking surface at a high level. **Chapter 14** for the model registry, the per-model matrix and the discipline that an unverified fact is UNKNOWN and must raise. **Chapter 15** for the (model, effort) pair as the unit of choice. **Chapter 16** for the request body and the typed content blocks. **Chapter 17** for streaming, where `thinking_delta` and `signature_delta` were introduced and deliberately left for this chapter to explain.

Chapter 13 asked what thinking *buys*. This chapter asks what the API will *accept*, which is a different and much less forgiving question. Prompt caching gets its own full treatment in Chapter 30; tools get Part 6. What is here is thinking as an API surface, and the several ways it fails quietly.

Introduction: The Migration That Passed Every Test

A team moves an agent from Opus 4.6 to Opus 5. One line changes:

```
model="claude-opus-5",
thinking={"type": "enabled", "budget_tokens": 12000},
```

Every unit test passes, because every unit test mocks the API. Staging is fine, because staging still points at 4.6. Production returns a 400 on the first request: **extended thinking is legacy, and Claude 4.7 and later reject `type: "enabled"` outright.**

That one is loud, and it is fixed in ten minutes. The three that follow are not.

The team switches to `{"type": "adaptive"}` and the agent works. A week later the bill for that agent has roughly **tripled** on long sessions, and nobody can find the extra tokens in the output. They are not in the

output. Opus 5 keeps every prior turn's thinking in context, and on a twenty-turn tool-using session **76.6 per cent of every input token billed is a thinking block from a turn that already finished** — 1,140,000 tokens out of 1,489,200. Section 18.26 prints the table. Nothing was misconfigured. The default changed underneath them, and the default is the right one; it just has a price they had not counted.

Then a clever engineer adds an effort tuner: `low` for simple requests, `xhigh` for hard ones, chosen per request. Reasoning quality improves. Cost goes *up* by a third, because the resolved effort level is rendered into the prompt, so every change starts a new cache prefix. Section 18.27 measures it: **133.3 per cent dearer than a single fixed configuration** over twenty requests on a 40,000-token cached prefix.

And finally, an intermittent 400 in production that no fixture reproduces. The code echoes the assistant turn back with the tool result, filtering thinking blocks like this:

```
blocks = [b for b in content if b["type"] == "thinking"]
```

which silently drops every `redacted_thinking` block. Those appear only when part of the model's reasoning has been safety-redacted, which is a property of the model's output rather than of your code path. Section 18.24 measures a twelve-turn conversation in which one turn in four is redacted: **the correct filter round-trips 12 of 12; the naive one round-trips 9, and the first failure is turn 4.** It reproduces on a quarter of traffic and on none of your tests.

Four failures, one theme, and it is the theme of the whole chapter. Thinking is not a switch. It is a **configuration with per-model rules, a round-trip protocol, a context-window cost and a cache identity** — and three of those four fail silently. This chapter builds the preflight that catches them on your machine.

18.1 Two Modes, and Why One Is Legacy

There are exactly two ways to configure thinking, and which one a model accepts is a hard fact about that model.

Mode	Shape	Status
Adaptive	`{"type": "adaptive"}`	Current. The model decides whether and how deeply to think, per request.
Extended (manual)	`{"type": "enabled", "budget_tokens": N}`	**Legacy.** You name a token budget.

Adaptive is not "extended thinking with a better default". It is a different contract. Under extended thinking you buy a quantity of reasoning in advance; under adaptive, whether Claude thinks at all on a given request depends on your configuration *and the complexity of the request*. On a simple question an adaptive model may produce **no thinking block whatsoever**, regardless of `display`. Code that assumes `content[0]` is a thinking block is Chapter 16's bug wearing a different hat.

The per-model rules, verified 30 July 2026:

Model	Mode	Thinking on by default?
Fable 5, Mythos 5, Mythos Preview, Opus 5, Sonnet 5	Adaptive	**Yes**
Opus 4.8, Opus 4.7	Adaptive	No — set `{"type": "adaptive"}`
Opus 4.6, Sonnet 4.6	Adaptive **or** extended	No
Opus 4.5, Sonnet 4.5, Haiku 4.5	**Extended only**	No

Two consequences worth carrying. **Extended thinking still exists and is still the only mode on the 4.5 generation, including Haiku 4.5** — so "extended thinking is legacy" does not mean "delete that code path" if you serve a mixed fleet. And **on the current 5-series models thinking is already on**, which means the first thing most developers need is not how to enable it but how to *see* it.

18.2 Turning It Off, Where You Can

`{"type": "disabled"}` is the third value, and it is the one with the most per-model variation in the whole API.

Model	`thinking: {"type": "disabled"}`
Sonnet 5	Accepted.
Opus 5	Accepted at effort `high` **or below**; **400 at** `xhigh` **or** `max`.
Fable 5, Mythos 5, Mythos Preview	**Rejected.** Thinking cannot be turned off.
Opus 4.8, 4.7, 4.6, Sonnet 4.6	Off by default; the registry in this book records the disable rule as **UNKNOWN** and raises rather than guessing.

That last row is the book's discipline showing its teeth. Chapter 14's rule was that an unverified fact is UNKNOWN and that UNKNOWN must raise rather than default. The documentation states the disable rules for the 5-series explicitly and does not state them for the 4.6 generation, where thinking is off unless you ask for it. So the preflight in section 18.19 **raises** `Unverified` if you ask it whether Opus 4.6 can disable thinking. A test asserts that it raises. Guessing `True` there would be a plausible, silent, wrong answer — exactly the species this chapter is about.

Two more rules that are easy to trip over:

- `display` **is invalid with** `disabled`**.** There is nothing to display. It is a 400, not an ignored field.
- **Opus 5 with thinking disabled can occasionally emit tool calls as plain text or leak internal XML tags into its visible output.** The documentation says so and points at prompting mitigations. If you turn thinking off on Opus 5 to save money, budget for output validation you did not previously need.

18.3 The Thinking Block

A response with thinking has one or more `thinking` blocks **before** the `text` blocks:

```
{"content": [
  {"type": "thinking",
   "thinking": "Let me break this down. The question has two parts...",
   "signature": "WaUjzkypQ2mUEVM36O2Txu...."},
  {"type": "text", "text": "Based on my analysis..."}]}
```

Two fields, and they do very different jobs.

`thinking` is **a summary of Claude's reasoning, never the raw chain of thought.** No `display` setting returns the raw chain. The summary is produced by a **different model** from the one you targeted, and the thinking model does not see the summary. Summarisation behaviour is explicitly subject to change.

`signature` is an **encrypted copy of the full reasoning**. It is what the API uses to verify that a thinking block you send back was genuinely generated by Claude. It is opaque — do not parse it — it is much longer on Claude 4 and later models, and it is **compatible across platforms**: a signature generated on the Claude API works on Bedrock and on Google Cloud.

> WARNING
>
> **The billing sentence that catches everyone**
>
> **You are charged for the full thinking tokens generated, not for the summary you can see.** The billed output token count does *not* match the token count of the text in the response. If you are reconciling a bill against `len(block.thinking)`, you will conclude that Anthropic is overcharging you. You are reading a summary of something longer that you paid for.

18.4 `display`: Summarised, Omitted, and What Changes

`display` sits on the thinking configuration and works in both modes. It takes two values.

`display`	Thinking block contains	Default on
`"summarized"`	The summary text	Opus 4.6, Sonnet 4.6 and earlier
`"omitted"`	An **empty** `thinking` field; `signature` still present	Fable 5, Mythos 5, Opus 5, Sonnet 5, Opus 4.8, Opus 4.7, Mythos Preview

The reason to choose `omitted` is precise and worth stating exactly: **faster time-to-first-*text*-token when streaming.** Chapter 17 showed the mechanism — under `omitted` the server sends no `thinking_delta` events at all; the block opens, receives a single `signature_delta`, and closes. Text begins streaming sooner by the entire duration of the thinking summary.

What it does not change is the bill. **Omitting reduces latency, not cost.** You generated those tokens and you are paying for them either way.

Four smaller rules:

- Round-tripped omitted blocks are passed back unchanged; the server decrypts the `signature` to reconstruct the original thinking.

- Any text you write into the empty `thinking` field of a round-tripped omitted block is **ignored**, not rejected. This is the one documented exception to "complete and unmodified".
- Switching `display` between turns is supported, and the `signature` is identical either way.
- In the Ruby SDK the field is `display_:` with a trailing underscore, to avoid shadowing `Kernel#display`. The wire field is still `display`.

18.5 Effort Is Not Thinking

These are two parameters with two jobs, and conflating them is the most common conceptual error in this part of the API.

- `thinking` controls *whether* Claude reasons in thinking blocks before answering.
- `output_config.effort` controls *how much work Claude puts into the whole response* — text, tool calls and thinking together. In adaptive mode that includes how often and how deeply it thinks.

`effort` takes `low`, `medium`, `high`, `xhigh`, `max`. `high` **is the default** and is identical to omitting the parameter. `adaptive` **is never a valid effort value**; it is a thinking mode, and the documentation calls that mistake out by name.

The documented decision table is short enough to memorise:

Goal	Lever
Lower cost or latency on a thinking-enabled workload	Lower `effort` first — it scales the whole response down.
Claude thinks too rarely or too shallowly	Raise `effort`, or steer with the prompt.
Thinking fully off	`thinking: {"type": "disabled"}`, where the model allows it.
A hard ceiling on spend	`max_tokens`. Effort is soft guidance; `max_tokens` is a strict limit.

Per-model support is not uniform and does not follow the tier. `xhigh` is available on Fable 5, Mythos 5, Opus 5, Opus 4.8, Opus 4.7 and Sonnet 5; `max` additionally on Opus 4.6 and Sonnet 4.6. **Haiku 4.5 does not support** `effort` **at all** — not "defaults to high", not "ignores it": the parameter is unsupported. Chapter 15's running thread applies unchanged: **capability does not travel in a bundle — read the per-model matrix, never the tier.**

18.6 Task Budgets

`output_config.task_budget` is a beta control, behind the header `task-budgets-2026-03-13`, that tells the model roughly how many tokens the *whole task* should take — across an agentic loop, not a single response.

Fact	Value
Supported on	Opus 5, Fable 5, Mythos 5, Opus 4.8, Opus 4.7 — **not Sonnet 5**, and not Claude Code or Cowork

Fact	Value
Minimum `total`	**20,000** tokens; below that is a 400
Enforcement	**Advisory, not enforced.** The countdown is visible only to the model.

Read that last row twice, because it is the trap. A task budget is a *signal*, not a limit. An undersized budget does not truncate the task; it produces refusal-like behaviour, because the model believes it has been asked to do something in less room than it needs. If you want a limit, `max_tokens` is a limit.

18.7 Thinking with Tool Use: A Loop Is One Turn

Here is the sentence that reorganises how you think about agent loops:

A tool-use loop is one assistant turn. From the model's perspective the turn does not complete until Claude has finished its full response, however many tool calls that takes.

```
User:      "What's the weather in Paris?"
Assistant: [thinking] + [tool_use: get_weather]
User:      [tool_result: "20 C, sunny"]
Assistant: [text: "The weather in Paris is 20 C and sunny"]
```

Four messages, two API calls, **one assistant turn**. From that single fact, three rules follow.

One thinking mode per turn. You cannot toggle thinking in the middle of a turn, including inside the tool-use loop.

Mid-turn conflicts degrade gracefully — which is worse than an error. If you do toggle mid-turn, the API does not reject the request. It **silently disables thinking** for it, and may strip thinking blocks that would create an invalid turn structure. There is no error, no warning field, and no change to the response shape you can pattern-match on. The documented way to know whether thinking was active is to **check whether the response contains any thinking blocks**. Build that check; nothing else will tell you.

Toggle between turns, not within them. Decide your thinking strategy at the start of each turn, complete the turn, then change it. And note that changing it invalidates prompt caching — section 18.12.

One more constraint applies only to the legacy mode: **manual extended thinking supports `tool_choice: auto` or `none` only.** Forcing tool use with `any` or a named tool is an error, because forcing tool use is incompatible with manual extended thinking. **Adaptive thinking supports forced tool use.** If a migration to adaptive suddenly lets you force a tool, that is not a coincidence.

18.8 Preserving Thinking Blocks

When Claude calls a tool it pauses construction of its response. When you return the tool result, it continues building *that same response*, so the reasoning that led to the call must still be there. Pass every thinking block back **complete and unmodified**, alongside the `tool_use` block it accompanied.

The documented obligations, in the documentation's own three words:

	Rule
Required	Within a tool-use turn, pass thinking blocks back.
Recommended	Across turns, pass everything back.
Allowed	Outside tool use, omit prior turns' thinking.

You do not need to prune old thinking yourself. Pass it all back; the API filters it, keeps what preserves the model's reasoning, and **bills input tokens only for the blocks actually shown to Claude**. Which prior blocks are kept is per-model — section 18.11 — and the `clear_thinking_20251015` context-editing strategy overrides the default in either direction.

The hard constraint is narrow and absolute:

> WARNING
>
> **Within the latest assistant message**
>
> The sequence of consecutive thinking blocks must match what the model generated. **You cannot rearrange, edit, or partially drop them**, and that includes `redacted_thinking` blocks. Modified blocks are rejected with a 400. The one exception is text placed in the empty `thinking` field of an omitted block, which is ignored rather than rejected.

Note what "modified" does *not* mean. Reading the summary text and displaying it to a user is fine. The API rejects blocks whose returned content has been changed, not blocks you have read.

18.9 Redacted Thinking

Sometimes part of Claude's reasoning is safety-redacted. When that happens you get a different block type:

```
{"type": "redacted_thinking", "data": "EvQ...opaqueciphertext"}
```

`data` is opaque and encrypted, exactly like `signature`. It carries no readable text, and it must be passed back unchanged in the same way.

This is a distinct concept from `display: "omitted"`, and the two are constantly confused. Omitted display returns a **regular `thinking` block with an empty `thinking` field**. Redaction returns a `redacted_thinking` **block with a `data` field**. Same intent from your side — pass it back untouched — but a different `type`, which is precisely why the filter breaks:

```
blocks = [b for b in content if b["type"] == "thinking"]        # wrong
blocks = [b for b in content                                    # right
          if b["type"] in ("thinking", "redacted_thinking")]
```

The documentation flags this by name: filtering on `"thinking"` alone **silently drops** `redacted_thinking` blocks and breaks the multi-turn protocol. Section 18.24 measures what "breaks" costs.

18.10 Interleaved Thinking

Interleaved thinking lets Claude reason **between** tool calls — reasoning about one tool's result before deciding what to do next, rather than only before the first call.

- With **adaptive** thinking it is **automatic on every model that supports adaptive**. No beta header.
- On Fable 5, Mythos 5, Mythos Preview, Opus 5, Opus 4.8 and Opus 4.7, reasoning between tool calls **always** appears in thinking blocks.
- **Haiku 4.5 does not support it.**
- In manual extended mode it requires a beta header and changes how the budget is counted.

Two clarifications the documentation is careful to make, both of which correct a common belief. **Consecutive tool calls do not require interleaved thinking** — Claude can chain tool calls either way; interleaving changes *where* thinking appears, not *whether* chaining is possible. And **with interleaving, the thinking allocation can span the entire assistant turn** rather than a single response, which is the reason section 18.12 warns that interleaving amplifies cache invalidation.

18.11 Preservation by Model, and the Context Bill

Whether previous turns' thinking blocks stay in context **by default** depends on the model, and this single fact is the largest cost variable in the chapter.

Behaviour	Models
Keep all prior turns	Opus 4.5 and later Opus, Sonnet 4.6 and later Sonnet, Fable 5, Mythos 5, Mythos Preview
Keep the last turn only	Earlier Opus and Sonnet, and **all Haiku through 4.5**

On keep-last models the API strips older blocks automatically when you pass them back; you do not remove them yourself.

Preservation is documented as a good thing, and it is: it enables cache hits during tool use, because preserved blocks are passed back with tool results and cached incrementally across the turn, and it has **no negative effect on model performance**. The trade-off is stated just as plainly: **long conversations consume more context space, because retained thinking blocks count as input like any other history.**

Section 18.26 puts numbers on "more". On a twenty-turn tool-using session with 1,500 thinking tokens per call, a keep-all model bills **1,489,200 input tokens against 349,200** — and a 1M context window fills after **266 turns instead of 1,313**, a session **4.9 times shorter**.

> WARNING
>
> **Switching models mid-conversation**
>
> When you switch between any two models — after a classifier refusal fallback, for instance — **strip `thinking` and `redacted_thinking` blocks from prior assistant turns.** Thinking blocks are tied to the model that produced them. Other models **silently ignore them rather than rejecting the request, but ignored blocks still add input tokens.** Silence plus a bill is the worst combination in this book. Section 18.25 prices it.

18.12 Thinking and Prompt Caching

Three rules, and the first one is the expensive one.

The configuration is part of the prompt. The thinking configuration and the **resolved `effort` level** are rendered into the prompt itself, so changing any of them **starts a new cache prefix**. Switching between `adaptive`, `enabled` and `disabled`, changing `budget_tokens`, and changing the effort value all invalidate cache breakpoints — message-level breakpoints always miss, and tool and system-prompt breakpoints can miss too. Treat any thinking or effort change as starting the cache over. Setting a parameter explicitly to its default is equivalent to omitting it, so `effort="high"` and no `effort` share a prefix.

Thinking blocks are cached with tool results. During a tool-use loop, a follow-up request carrying tool results can cache the previous history *including its thinking blocks*, automatically, even without explicit `cache_control` markers. Those cached blocks then count as input tokens when read. The trade-off is exact: thinking you never see again still contributes to input usage.

Whether prior blocks are in context at all is per-model, per section 18.11. On keep-last models, once you send a user message that is not a tool result, all previous thinking is stripped from context and the conversation is processed as if it had never been there.

One operational note worth acting on: **thinking-heavy tasks often outlive the default five-minute cache lifetime.** The one-hour cache duration exists for exactly this, at a 2× write multiplier instead of 1.25×. Chapter 11's arithmetic tells you when that trade is worth taking.

18.13 Thinking and the Context Window

`max_tokens` includes **all thinking generated in the current turn** and is a strict limit. Set it high enough for both the reasoning and the answer.

On Claude 4.5 and newer, if input tokens plus `max_tokens` exceeds the context window, **the API accepts the request**; if generation then reaches the window limit it stops with `stop_reason`: `"model_context_window_exceeded"` — Chapter 16's seventh stop reason, in its natural habitat. On earlier models the API returns a validation error instead. The modern behaviour is friendlier and more dangerous: a request that would once have been refused now succeeds partially, and your `stop_reason` branch is the only thing that notices.

How thinking counts depends on when it was generated:

- **Current-turn thinking** counts toward `max_tokens`, is billed as **output**, and occupies window space for its turn.
- **Prior-turn thinking** depends on the preservation default. On keep-all models it stays in context, counts toward the window, and is billed as **input** like any other history. On keep-last models it is stripped and costs nothing.

So: **on keep-all models, budget your context window as if thinking were ordinary conversation history, because it is.** Use thinking-block clearing to reclaim space when a session runs long, and use the free `count_tokens` endpoint (Chapter 6) rather than estimating.

18.14 Limits and Feature Compatibility

The compatibility rules, consolidated. Every one of them is a preflight check in section 18.22.

Constraint	Rule
Sampling parameters	Non-default `temperature`, `top_p`, `top_k` return a **400 on every request** — thinking or not — on Fable 5, Mythos 5, Mythos Preview, Opus 5, Opus 4.8, Opus 4.7 and Sonnet 5. On older models the restriction applies only while thinking is on: `temperature` and `top_k` are incompatible; `top_p` is allowed between **0.95 and 1**.
Prefill	You cannot prefill the assistant response while thinking is on. Separately (Chapter 16), prefilling is unsupported on 4.6 and later regardless.
Forced tool use	Incompatible with manual extended thinking; works with adaptive.
Output limits	128K on Fable 5, Mythos 5, Mythos Preview, Opus 5, Opus 4.8, Opus 4.7, Sonnet 5, Opus 4.6, Sonnet 4.6. **64K** on Haiku 4.5, Sonnet 4.5, Opus 4.5. Batch API raises it to 300K via `output-300k-2026-03-24` on Opus 5, Opus 4.8, Opus 4.7, Sonnet 5, Opus 4.6, Sonnet 4.6.
Long requests	SDKs require streaming above `max_tokens` **21,333** (Chapter 17). Above roughly **32,000 thinking tokens per request**, the documentation recommends the **Batch API**, because such requests run long enough to hit system timeouts and open-connection limits.

One more, specific to Fable 5 and Mythos 5: the raw chain of thought is **never** returned, and a request that tries to elicit the model's internal reasoning as part of the response *text* can be refused with `stop_details.category: "reasoning_extraction"`. Chapter 16 established that a refusal is an HTTP 200 carrying a category; this is a category you can provoke by asking the wrong way. The supported way to see reasoning is to read the thinking blocks.

18.15 Visual Analogies

VISUAL ANALOGY

Thinking is a working, not a rough draft

A mathematician's working is not a worse version of the proof; it is a different artefact with a different purpose. You keep it because the next step depends on it, and because if the proof is wrong the working is where the error is. The `signature` is the examiner's stamp: it certifies that this working came from this candidate, which is why you may photograph it but not rewrite it.

Where it breaks. A mathematician can re-read their own working; the model cannot introspect its own reasoning (Chapter 10's no-introspection limit is unrepealed by any of this). And what you are shown is a *summary written by someone else* — a different model — not the working itself. Nobody's rough paper is transcribed by a second mathematician before you are allowed to see it.

VISUAL ANALOGY

Effort is a brief; max_tokens is a budget

Telling a contractor "do a thorough job" is a brief. Telling them "you have twelve thousand pounds" is a budget. The brief changes how they work; the budget stops them. `effort` is the brief — soft guidance, applied across text, tool calls and thinking alike. `max_tokens` is the budget, and it is enforced. `task_budget` is a third thing: a brief phrased as a budget, advisory, visible only to the contractor.

Where it breaks. A contractor given too small a budget stops early and tells you. An undersized `task_budget` produces refusal-like behaviour instead — the model behaves as though the job cannot be done, rather than doing part of it. The failure is a change of manner, not a partial delivery.

VISUAL ANALOGY

A tool-use loop is one sentence with an interruption

"I'll check the weather —" *[waits]* "— and it's twenty degrees." Two API calls, one sentence. You would not expect to change your accent halfway through, and the API will not let you change thinking mode halfway either. If you try, it does not stop you; it quietly finishes the sentence in the other voice.

Where it breaks. A speaker knows they were interrupted. Your code does not, because graceful degradation leaves no error and no flag — only the absence of thinking blocks in a response you were not inspecting.

18.16 Architecture: One Thinking Policy, Not Fifty Call Sites

The failure mode this architecture prevents is the one in the introduction: a thinking configuration assembled inline at each call site, so that no single place knows what the current configuration is, and nothing can compare it against the model's rules.

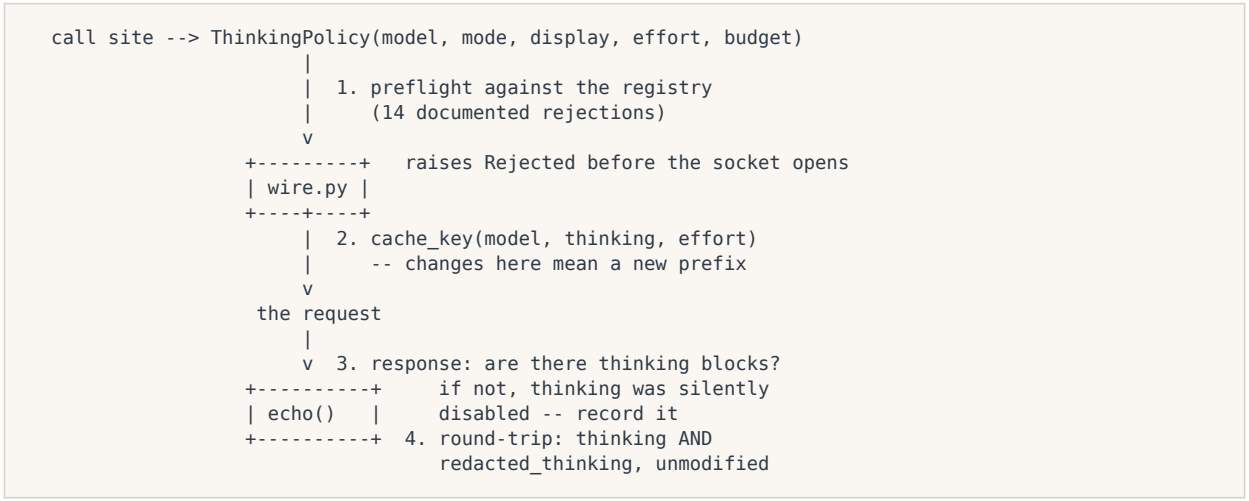


Figure 18.1 — A thinking policy object

Four responsibilities, one object:

1. **Preflight.** Every documented rejection, checked offline.
2. **Cache identity.** A policy knows its own cache key, so a change is a decision somebody makes rather than a side effect.
3. **Degradation detection.** After every call, assert that thinking blocks are present when the policy says they should be. This is the only defence against silent mid-turn disabling.
4. **Round-trip.** One function that echoes an assistant turn, and it knows about both block types.

If `thinking={` appears in more than one file in your codebase, you have as many thinking policies as you have call sites, and no way to reason about the cache.

18.17 Examples: Four Configurations

A. See the reasoning on a 5-series model. Thinking is already on; `display` is what you are changing.

```
{"model": "claude-opus-5", "max_tokens": 16000,
 "thinking": {"type": "adaptive", "display": "summarized"},
 "messages": [{"role": "user", "content": "Why does this test flake?"}]}
```

B. Turn thinking on for a 4.6-generation model, which has it off by default.

```
{"model": "claude-opus-4-6", "max_tokens": 16000,
 "thinking": {"type": "adaptive", "display": "summarized"}}
```

C. Cheapest legal configuration for a bulk summariser. Note the model: Sonnet 5 allows `disabled`; Fable 5 would 400.

```
{"model": "claude-sonnet-5", "max_tokens": 1024,
 "thinking": {"type": "disabled"},
 "output_config": {"effort": "low"}}
```

D. A long agentic task with a budget signal. `task-budgets-2026-03-13` in `anthropic-beta`, `total` at or above 20,000, and streaming because `max_tokens` is over 21,333.

```
{"model": "claude-opus-5", "max_tokens": 64000, "stream": true,
 "thinking": {"type": "adaptive"},
 "output_config": {"effort": "xhigh",
                   "task_budget": {"total": 200000}}}
```

Change `effort` in D between requests and you have thrown away the cache. Change it to `adaptive` and you have a 400. Both are in the preflight.

18.18 Code Walkthrough: A Preflight That Refuses to Guess

The mini project extends Chapter 14's `registry.py` and Chapter 16's `wire.py`. It adds no new facts; it makes the facts the registry already holds *executable*.

The preflight is one function returning a list of reasons, not a boolean:

```
def preflight(model_name, *, thinking=None, effort=None,
              task_budget=None, max_tokens=1024, stream=False,
              tool_choice=None, sampling=None, prefill=False,
              betas=()):
    m = registry.get(model_name)
    bad = []
    def no(rule, why):
        bad.append((rule, why))
```

A list rather than a boolean because a request can be wrong in several ways at once, and a validator that stops at the first one turns a single fix into four deploys. The tagged `rule` string is what tests assert on; the `why` string is what a human reads.

The mode check reads the registry rather than a hard-coded table:

```
supported = m.get("thinking")          # adaptive | extended | both
if mode == "enabled" and supported == "adaptive":
    no("legacy-extended-rejected", ...)
if mode == "adaptive" and supported == "extended":
    no("adaptive-unsupported", ...)
```

`m.get(...)` is Chapter 14's accessor, not a dictionary lookup. It raises `Unverified` if the field was never verified. That is why the disable check is written the way it is:

```
if mode == "disabled":
    can = m.get("can_disable")     # True | False | 'effort<=high'
    if can is False:
        no("disable-rejected", ...)
    elif can == "effort<=high" and effort in ("xhigh", "max"):
        no("disable-at-high-effort", ...)
```

Ask this preflight whether Opus 4.6 can disable thinking and it **raises**, because the documentation states the disable rules for the 5-series and does not state them for the 4.6 generation. A test asserts the raise. It would have been easy, and plausible, and wrong, to write `can = m.can_disable or False`.

The sampling check is the one that encodes a per-generation split rather than a per-model flag:

```
if m.get("sampling_locked"):
    no("sampling-locked", "... on EVERY request, thinking or not")
elif mode in ("adaptive", "enabled"):
    for p in sorted(sampling):
        if p in ("temperature", "top_k"):
            no("sampling-with-thinking", ...)
        elif p == "top_p" and not 0.95 <= sampling[p] <= 1.0:
            no("top-p-out-of-range", ...)
```

On Opus 5 a `temperature` is a 400 whether or not thinking is on. On Opus 4.6 it is a 400 only while thinking is on, and `top_p` between 0.95 and 1 is fine. Two different rules that look like one rule, which is exactly the kind of thing a preflight should own so that nobody has to remember it.

Then the round trip, which is four lines of substance and one of the most consequential functions in the book:

```
THINKING_BLOCK_TYPES = ("thinking", "redacted_thinking")
def thinking_blocks(content):
    return [b for b in content if b.get("type") in THINKING_BLOCK_TYPES]
def naive_thinking_blocks(content):
    return [b for b in content if b.get("type") == "thinking"]
```

Both are kept, and the naive one is kept **on purpose**, exactly as Chapter 16 kept `content[0].text` as a named function. A test asserts that it fails. When somebody deletes it as dead code, the suite will tell them they have removed a demonstration rather than found a bug.

`round_trip_ok` then checks what the API checks — same count, same order, same `signature`, same `data` — with the single documented exception encoded explicitly:

```
if x.get("thinking") and x.get("thinking") != y.get("thinking"):
    return False, "thinking text modified"
```

The `if x.get("thinking")` guard is the exception. If the original block's `thinking` field was empty — an omitted block — then whatever you wrote into it is ignored rather than rejected, and this comparison correctly declines to care.

18.19 Mini Project: The Thinking Preflight and the Context Accountant

What you are building. Two things that share a registry: a validator that reproduces every documented thinking rejection offline, and an accountant that prices the one thinking behaviour nobody configures — preservation.

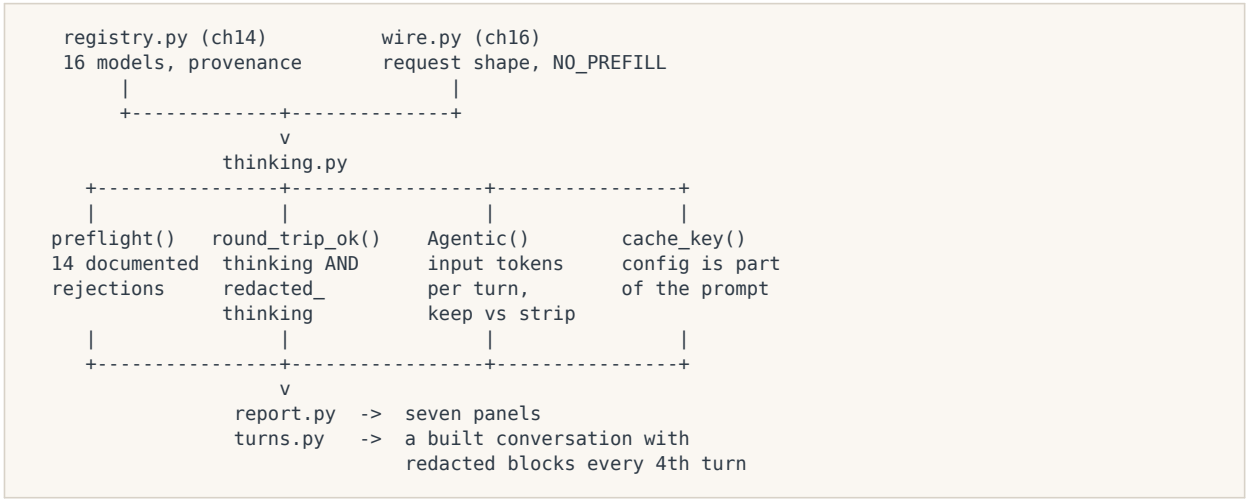


Figure 18.2 — ch18/ architecture

Folder structure.

```
ch18/
  registry.py       copied from ch14/  -- unchanged
  wire.py           copied from ch16/  -- unchanged
  thinking.py       preflight, round trip, accountant, cache keys
  turns.py          a constructed agentic conversation
  report.py         every number in this chapter
  test_thinking.py  38 tests
```

`turns.py` declares its provenance in the Chapter 17 style: **BUILT**, with block shapes transcribed from the documentation on 30 July 2026 and the conversation around them constructed. No fixture in this book pretends to be a recording it is not.

Run it.

```
cd ch18
python3 report.py
python3 -m pytest -q      # 38 passed
```

18.20 Panel A — The Matrix, and Three Cells That Refuse to Answer

```
model              mode     on?   disable      display    prior inter eff budgets
claude-fable-5     adaptive True  False        omitted    all   auto  5   True
claude-opus-5      adaptive True  effort<=high omitted    all   auto  5   True
claude-sonnet-5    adaptive True  True         omitted    all   auto  5   False
claude-opus-4-8    adaptive False UNKNOWN      omitted    all   auto  5   True
claude-opus-4-6    both     False UNKNOWN      summarized all   auto  4   False
claude-sonnet-4-6  both     False UNKNOWN      summarized all   auto  4   False
claude-haiku-4-5   extended False True         summarized last  none  -   False
3 of 49 cells are UNKNOWN and RAISE rather than default.
Chapter 14's rule, still holding: an unverified fact is not a False.
```

Seven models, seven properties, and not one column where the answer follows from the model's tier. Sonnet 5 can disable thinking and Fable 5 cannot. Opus 5 supports task budgets and Sonnet 5 does not.

Haiku 4.5 is the only extended-only model here, has no effort parameter at all, is the only one that keeps just the last turn's thinking, and is the only one with no interleaving.

The three UNKNOWN cells are the point of the panel. They are not gaps to be filled with a reasonable guess; they are the registry declining to lie. Ask for one and you get an exception naming the field and telling you to check the documentation.

18.21 Panel B — Fourteen Documented Rejections

```
  FAIL  legacy extended on a 5-series model  legacy-extended-rejected
  FAIL  adaptive on an extended-only model   adaptive-unsupported
  FAIL  enabled without budget_tokens        budget-required
  FAIL  disable on Fable 5                   disable-rejected
  FAIL  disable at xhigh on Opus 5           disable-at-high-effort
  FAIL  display alongside disabled           display-with-disabled
  FAIL  effort on Haiku 4.5                  effort-unsupported
  FAIL  xhigh on Opus 4.6                    effort-level-unsupported
  FAIL  'adaptive' passed as an effort       adaptive-is-not-an-effort
  FAIL  task_budget on Sonnet 5              task-budget-unsupported
  FAIL  task_budget below the floor          task-budget-too-small
  FAIL  temperature on Opus 5                sampling-locked
  FAIL  forced tools with manual thinking    forced-tools-manual-thinking
  FAIL  max_tokens 60000 without streaming   streaming-required
14 of 14 rejected before a single byte left the machine.
```

Every one of these is a 400 you would otherwise discover in production, and every one is a line in the documentation rather than a discovery of ours. The value is not the cleverness; it is that a rule written in prose on a web page is now a test that runs in a tenth of a second on a laptop with no API key.

Note the fourth and fifth rows together. Turning thinking off is legal on Sonnet 5, illegal on Fable 5, and legal-or-illegal on Opus 5 **depending on a different parameter**. A validator is the only sane place for a rule shaped like that.

18.22 Panel C — Nine Requests a Team Would Plausibly Write

```
  400  port a 4.6 config to Opus 5           legacy-extended-rejected
   ok  cheap summariser, thinking off
  400  cheap summariser on Fable 5           disable-rejected
  400  max reasoning, thinking off to save money disable-at-high-effort
  400  Haiku with effort=low to cut cost     effort-unsupported
  400  long report, no stream                streaming-required
   ok  long report, streamed
   ok  agent with forced tool choice
  400  temperature 0 for determinism         sampling-locked
6 of 9 fail preflight, offline, with no key.
```

Six of nine. Chapter 14 ran the same exercise on model IDs and got eight of nine; this is the same lesson about a different surface, and it is worth being explicit about why the hit rate is so high. These are not contrived. Every one of them is a reasonable thing to write:

- **Porting a 4.6 config** is what a migration *is*.

- **"Turn thinking off to save money"** is the correct instinct and it is model-dependent, twice over — Fable 5 refuses outright, and Opus 5 refuses at `xhigh`.
- **"Use effort=low on the cheap model"** is exactly the optimisation Chapter 15 taught, applied to the one current model that has no effort parameter.
- `temperature=0` **for determinism** is the single most ingrained habit in the industry and it is now a 400 on every 4.7-and-later model on every request.

Four of the six failures are someone doing the *right thing* on the *wrong model*. That is what the per-model matrix is for, and it is why "read the matrix, never the tier" keeps earning its place in this book.

18.23 Panel D — The One-Word Bug

```
  correct  accepted 12 of 12  first failure: None
  naive    accepted  9 of 12  first failure: (4, 'dropped 1 of 2 thinking blocks')
  correct : [b for b in content
             if b['type'] in ('thinking', 'redacted_thinking')]
  naive   : [b for b in content if b['type'] == 'thinking']
```

A twelve-turn tool-using conversation in which every fourth assistant turn contains a `redacted_thinking` block alongside its regular one. The correct filter round-trips all twelve. The naive filter round-trips nine and fails on turns 4, 8 and 12 with a 400 that says the thinking blocks were modified.

Three properties make this the nastiest bug in the chapter.

It is a function of the model's output, not of your code path. You cannot reach it by taking a different branch. It appears when the model's reasoning happens to be safety-redacted, which no test of yours controls.

It reproduces on a fraction of traffic. Twenty-five per cent here, because that is how the fixture was built; in production, whatever the redaction rate happens to be that week. Enough to page someone, too little to bisect.

Every fixture you already have passes. If your test conversation was recorded on a day when nothing was redacted, your suite is green and your production is not.

The fix is one tuple. The lesson is more general and worth stating plainly: **when a protocol has two block types that must be treated identically, enumerate them in a named constant, never in a literal at the point of use.** `THINKING_BLOCK_TYPES` exists so that the day a third type is added, there is one place to add it.

18.24 Panel E — What a Model Switch Leaves Behind

```
12 turns of preserved thinking = 36,000 input tokens.
After a mid-conversation model switch the new model IGNORES
them -- silently, without an error -- and still bills them:
  once             $0.1800
  on 8 more turns  $1.4400
No exception, no warning, no behaviour change. Just a bill.
```

This one is small in absolute terms and important in kind. A refusal fallback, a cost-driven downgrade, an A/B test — anything that changes the model mid-conversation — leaves the previous model's thinking blocks in the history. The new model does not reject them. It does not use them either. It reads them as input, charges you for them, and discards them.

£1.44 on one twenty-turn session is nothing. On a fleet, it is a line item nobody can explain, because the tokens are real, the request is valid, and the behaviour is unchanged. It is the purest example in the book of a failure with no signal at all: **not an error, not a wrong answer, just money.**

The fix is a single rule attached to model selection rather than to message construction: *when the model changes, strip* `thinking` *and* `redacted_thinking` *from prior assistant turns.* Put it in the same object that chose the model, or it will not happen.

18.25 Panel F — The Context Accountant

```
 turns  input (keep) input (strip)     carried   share    $ keep   $ strip
     1         3,020         3,020           0    0.0%     0.098     0.098
     5        90,300        30,300      60,000   66.4%     0.864     0.564
    10       368,600        98,600     270,000   73.3%     2.668     1.318
    20     1,489,200       349,200   1,140,000   76.6%     9.096     3.396
    40     5,986,400     1,306,400   4,680,000   78.2%    33.232     9.832
At 20 turns, 76.6% of every input token billed is a thinking block from a turn
that already finished. The session costs $9.096 instead of $3.396.
Capacity, not money: the largest single request at 20 turns is
  keep-all  73,900 tokens
  stripped  16,900 tokens
and a 1M context window fills after
  266 turns keeping thinking
  1313 turns stripping it (4.9x longer)
```

The model is a tool-using agent on Opus 5: 1,500 thinking tokens per call, two calls per turn, a 400-token tool result, a 300-token answer, a 500-token system prompt. Nothing exotic; if anything the thinking figure is conservative for a hard task at `high` effort.

Read the `share` column down. At turn 5, two-thirds of what you are paying for on the input side is reasoning from earlier turns. At turn 20, more than three-quarters. At turn 40, seventy-eight per cent. **The dominant term in a long agentic conversation's input bill is not the conversation. It is the thinking.**

This is Chapter 16's running thread — *statelessness is quadratic: the conversation is a list you resend* — with a term nobody put in the list. The list you resend contains the model's own working, and on a keep-all model it contains all of it.

Two honest qualifications, because the documentation makes both and this book does not publish half a finding. **Preservation is not a bug and turning it off is not free.** Preserved thinking blocks enable cache hits during tool use, so the *cached* cost of a keep-all session is much lower than the $9.096 above; that figure is the uncached ceiling. And preservation has **no negative effect on model performance** — the trade is context and money against nothing at all in quality. The point is not "strip your thinking". The point is that this is a decision, it has a price, and until you have run this table for your own workload you have not made it.

The capacity figures are the ones to take to a design review. A 1M context window sounds inexhaustible until you notice that carried thinking shortens the session that fits inside it by a factor of **4.9**. If your agent needs to run for hundreds of turns, thinking-block clearing is not an optimisation; it is the feature that makes the session possible.

18.26 Panel G — The Configuration Is Part of the Prompt

```
20 requests over a 40,000-token cached prefix:
  one fixed configuration   $0.6900
  a five-level effort tuner $1.6100  (133.3% dearer)
  no caching at all         $5.0600
The tuner recovers 68.2% of the uncached cost where a fixed
configuration recovers 86.4%.
```

An engineer builds a per-request effort tuner: `low` for simple questions, `xhigh` for hard ones, five levels in rotation. Every individual decision is defensible. Cost goes up 133 per cent against leaving `effort` alone, because the resolved effort level is rendered into the prompt and therefore *is* part of the cache prefix. Five configurations means five prefixes, five cache writes at 1.25×, and reads that only ever hit one-fifth of the traffic.

The general statement is worth memorising because it generalises past this parameter: **anything the server renders into the prompt is part of your cache key, whether or not it looks like content.** The thinking mode, the `display` value, `budget_tokens`, and the resolved effort level all qualify. Setting a parameter explicitly to its default does not, which is why `effort="high"` and omitting `effort` share a prefix — a small mercy the test suite pins down.

There is a real trade here and it is not "never tune effort". It is that **effort tuning must be coarse.** Two levels chosen by a cheap classifier and held stable across a session will pay for themselves. Five levels chosen per request will not. Chapter 30 develops the arithmetic properly; the number you need today is that a cache prefix has an identity, and the thinking configuration is part of it.

18.27 Evaluation and Deployment

Thirty-eight tests. The ones worth naming: every one of the fourteen documented rejections fires; `assert_ok` names *all* the reasons rather than the first; asking for an unverified field raises `Unverified` rather than returning a default; the naive filter is asserted to accept exactly 9 of 12 turns; text written into an omitted block's empty `thinking` field is asserted to be *accepted*, encoding the one documented exception; and `effort="high"` is asserted to produce the same cache key as omitting `effort`.

To deploy a thinking policy safely:

- **Preflight in CI, not at runtime.** These are static facts about a model; the check belongs where a failing build stops a deploy.
- **Assert on the presence of thinking blocks** after every call where your policy expects them. That is the only detector for silent mid-turn disabling.
- **Emit the cache key as a metric.** A histogram of distinct cache keys per session tells you instantly whether something is churning your prefix.

- **Log `stop_reason` histograms** including `model_context_window_exceeded`, which is how the context accountant's arithmetic announces itself in production.
- **Re-verify the matrix on a schedule.** Chapter 14's `GET /v1/models` returns a `capabilities` object; use it to reconcile the checked-in registry, not to replace it.

18.28 Hands-on Labs

Lab 1 — Break the migration. Write the Opus 4.6 request from Example B, change only the model to `claude-opus-5`, and run it through `preflight`. *Success criterion:* you get `legacy-extended-rejected`, and you can say in one sentence why a mocked unit test would not have caught it.

Lab 2 — Ask an unanswerable question. Call `preflight("claude-opus-4-6", thinking={"type": "disabled"})`. *Success criterion:* it raises `Unverified`, and you can argue why raising beats returning `False`.

Lab 3 — Reproduce the redaction bug. Change `redacted_every` in `turns.py` from 4 to 12 and rerun the round-trip report. *Success criterion:* the naive filter now accepts 11 of 12, and you can explain why a lower failure rate is *worse* news.

Lab 4 — Price your own agent. Instantiate `Agentic` with your real thinking-token estimate, tool-result size and turn count. *Success criterion:* a percentage for carried thinking and a judgement about whether to enable thinking-block clearing.

Lab 5 — Find your cache churn. List every place your codebase sets `thinking` or `effort`, compute `cache_key` for each, and count the distinct keys. *Success criterion:* a number, and a plan if it is greater than two.

18.29 Lab Solutions

Lab 1. `[('legacy-extended-rejected', ...)]`. A mocked unit test asserts what your code does with a *response*; it cannot assert what the API does with a *request*. The only cheap defences are a preflight that encodes the rules or an integration test that spends money.

Lab 2. It raises. Returning `False` would mean "this model cannot disable thinking", which is a claim nobody has verified; returning `True` would mean the opposite, equally unverified. A third answer — "I do not know, and here is where to look" — is the only honest one, and it is the only one that produces a stack trace pointing at the exact field.

Lab 3. The naive filter accepts 11 of 12 and fails only on turn 12. That is worse, not better: a bug at 8 per cent is a bug that survives a canary, survives a week of monitoring, and is discovered by a customer. Failure rate and severity are independent, and low-rate failures are harder to fix precisely because they are hard to reproduce.

Lab 4. Run `Agentic(model, thinking_tokens=..., tool_result=..., answer=...)` and read `thinking_carried(turns) / input_tokens(turns)`. Enable clearing if the share exceeds roughly half *and* your sessions are long enough that the cached path does not rescue you. Note that clearing trades away cache hits during tool use, so measure both.

Lab 5. Two is a defensible answer: one configuration for interactive work and one for batch. Five is a cache invalidation with a reasoning-quality argument attached. If the number is greater than five, you do not have a policy; you have habits.

18.30 Best Practices

BEST PRACTICE

Fourteen habits

1. **One thinking policy object.** If `thinking={` appears in two files, you have two policies and no cache.
2. **Preflight in CI.** Fourteen rules, no API key, a tenth of a second.
3. **Read the per-model matrix, never the tier.** Sonnet 5 and Fable 5 disagree about whether thinking can be turned off.
4. **Never guess an unverified field.** UNKNOWN raises.
5. **Set `max_tokens` for thinking *and* answer**, and remember it is the only hard ceiling.
6. **Lower `effort` before you switch models** — the documentation's own advice, and Chapter 15's.
7. **Never pass `adaptive` as an effort value.** It is a mode.
8. **Echo `thinking` and `redacted_thinking` together**, from a named constant.
9. **Never edit a thinking block.** Reading it is fine; rewriting it is a 400.
10. **Strip thinking blocks when the model changes.** Attach the rule to model selection.
11. **Assert thinking blocks are present** when your policy expects them; silent disabling has no other signal.
12. **Keep the thinking configuration stable within a session** so the cache prefix survives.
13. **Consider the one-hour cache** for thinking-heavy work that outlives five minutes.
14. **Use the Batch API above roughly 32,000 thinking tokens per request**, as the documentation advises.

18.31 Common Mistakes

COMMON MISTAKE

Porting `budget_tokens` forward

Symptom. A 400 on the first production request after a model upgrade; every test green. **Cause.** `thinking: {"type": "enabled"}` is legacy and is rejected on Claude 4.7 and later. **Fix.** `{"type": "adaptive"}`, and steer with `effort` instead of a budget. **Detection.** Preflight in CI. Mocked tests cannot see request-side rules.

COMMON MISTAKE

Filtering thinking blocks on one type

Symptom. Intermittent 400s saying thinking blocks were modified, on a minority of traffic, reproducing on no fixture. **Cause.** `b["type"] == "thinking"` silently drops `redacted_thinking`. **Fix.** Enumerate both in a named constant. **Detection.** A fixture with a redacted block in it. Section 18.23 fails 3 of 12 without one.

COMMON MISTAKE

Toggling thinking inside a tool-use loop

Symptom. Reasoning quality drops on some turns; no errors anywhere. **Cause.** A tool-use loop is one assistant turn. Toggling mid-turn causes the API to **silently disable** thinking for that request. **Fix.** Decide at the start of a turn; change only between turns. **Detection.** Assert the presence of thinking blocks in responses where the policy expects them.

COMMON MISTAKE

Tuning effort per request

Symptom. Cost rises after a change that was supposed to reduce it. **Cause.** The resolved effort level is rendered into the prompt, so each level is a distinct cache prefix. Section 18.26: 133.3 per cent dearer over twenty requests. **Fix.** Coarse tuning, stable within a session. **Detection.** Count distinct cache keys per session.

COMMON MISTAKE

Reconciling the bill against the visible thinking text

Symptom. A conviction that output tokens are being over-counted. **Cause.** You are billed for the **full** thinking tokens; what you see is a summary produced by a different model. **Fix.** Use `usage.output_tokens_details.thinking_tokens`, not string length. **Detection.** Compare the two on any thinking response; they will not match.

COMMON MISTAKE

Assuming a thinking block is always there

Symptom. `IndexError` or a `KeyError` on `content[0]` on simple requests. **Cause.** Under adaptive thinking the model may skip thinking entirely, and then no block is produced regardless of `display`. **Fix.** Chapter 16's rule, unchanged: iterate typed blocks; never index. **Detection.** Send a trivial prompt to a 5-series model and inspect `content`.

COMMON MISTAKE

Setting `temperature=0` for determinism

Symptom. 400 on every request to a current model. **Cause.** Non-default sampling parameters are rejected on Claude 4.7 and later, thinking or not. **Fix.** Omit them and steer with prompting; use structured outputs where the shape must be fixed. **Detection.** It is in the preflight, and it is the most common single failure in Panel C.

COMMON MISTAKE

Treating `task_budget` as a limit

Symptom. A model that refuses plausible work, or overruns the budget entirely. **Cause.** Task budgets are **advisory**. The countdown is visible only to the model. **Fix.** `max_tokens` for a ceiling; `task_budget` only as a signal, never below 20,000. **Detection.** Compare actual usage against the budget over a week; the gap is the point.

18.32 Performance Considerations

- **Thinking adds latency, always.** Generating thinking blocks is generating tokens, and Chapter 11's finding stands: decode dominates. Expect longer responses when thinking is active.
- `display: "omitted"` **improves time-to-first-text-token** by the whole duration of the thinking summary, and changes nothing else. Latency, not cost.
- `effort` **is the throttle that moves everything at once** — text, tool calls and thinking. It is the first lever to reach for, ahead of changing model.
- **Above roughly 32,000 thinking tokens per request, move to the Batch API.** Such requests run long enough to hit system timeouts and open-connection limits, and the Batch API is 50 per cent cheaper besides.
- **Preservation is a latency asset and a context liability.** It enables cache hits across a tool-use turn, which shortens prefill; it also grows the prefix that has to be read. Section 18.25 quantifies the liability side.
- **The one-hour cache exists for this workload.** A 2× write against a 1.25× write pays for itself the moment a thinking-heavy task outlives five minutes.

18.33 Security Considerations

- **Thinking blocks are model output and may contain anything the model reasoned about**, including content you would not show a user. `display: "omitted"` is the safe default for any surface a customer can see, and it is already the default on current models.
- **Never log raw thinking text into a system with a lower classification than the conversation.** The summary is still derived from the full reasoning.
- `signature` **and** `data` **are opaque.** Do not parse, truncate, compress or "normalise" them. Signature values are long; storage systems that silently truncate long strings will produce 400s that look like tampering, because from the API's point of view they are.
- **Do not use thinking as an audit trail.** It is a summary written by a different model, explicitly subject to change, and Chapter 10's no-introspection limit means it is not a faithful account of the computation. Log inputs, outputs and tool calls.
- **Do not prompt for internal reasoning in the response text.** On Fable 5 that can be refused with `stop_details.category: "reasoning_extraction"`. Read the thinking blocks instead.
- **Strip thinking on model switch** — a leftover block from another model is data crossing a boundary for no benefit, and you are paying to carry it.

18.34 Real Production Examples

A migration that went well. A team moving from Opus 4.6 to Opus 5 wrote the preflight first, ran it against every request shape in their codebase, and found four incompatibilities in an afternoon: the legacy `budget_tokens`, a `temperature=0` inherited from 2023, a forced `tool_choice` that had been working around manual thinking, and a `max_tokens` of 32,000 with no `stream`. Total production incidents: zero.

An agent that ran out of room. A research agent designed for "up to 200 steps" started stopping at around 170 with `model_context_window_exceeded`. The cause was carried thinking; the fix was `clear_thinking_20251015` on turns older than ten, which restored the step budget at the cost of some cache hits on old prefixes.

A support product that lost its reasoning. Quality dropped intermittently after a refactor that set `thinking` per API call rather than per turn — so the second call of a tool-use loop sometimes disagreed with the first. The API never complained; it silently disabled thinking for those requests. The detector that found it was three lines: assert that a response contains a thinking block when the policy says it should.

A bill nobody could explain. A fallback path switched model on refusal and left the previous model's thinking blocks in the history for the rest of the session. No errors, no behaviour change, a few pounds per thousand sessions. Found by diffing input token counts against a locally computed estimate — which is the only way this class of failure is ever found.

18.35 Certification Notes

Concept	Exam domain	Why it is examinable
Adaptive vs legacy extended thinking	D1 Agentic Architecture & Orchestration	The migration fact of the current generation.
Per-model disable rules	D1	Capability does not travel in a bundle.
`effort` vs `thinking` vs `max_tokens`	D4 Prompt Engineering & Structured Output	Three controls, three jobs.
`display` changes latency, not cost	D5 Context Management & Reliability	A cost/latency distinction the exam likes.
A tool-use loop is one assistant turn	D1	Reframes the whole agent loop.
Preserving thinking blocks unmodified	D2 Tool Design & MCP Integration	The multi-turn tool protocol.
`redacted_thinking` as a distinct type	D2	The silent-failure classic.
Preservation by model	D5	Context budgeting for long sessions.
Configuration is part of the cache prefix	D5	Caching correctness.

Concept	Exam domain	Why it is examinable
Task budgets are advisory	D1	Advisory versus enforced.
Sampling parameters rejected on 4.7+	D4	A habit the platform has retired.
Silent degradation on mid-turn toggles	D5	Detection without an error signal.

18.36 Twenty Practice Questions

1. Name the two thinking modes and give the exact JSON for each.
2. Which models have thinking **on by default**?
3. What happens if you send `{"type": "enabled", "budget_tokens": 8000}` to Opus 5?
4. On which model can thinking be disabled at `high` effort but not at `xhigh`?
5. Which current models refuse `{"type": "disabled"}` outright?
6. What is in a thinking block's `signature` field?
7. Is the text in a `thinking` block the raw chain of thought?
8. What does `display: "omitted"` reduce?
9. What is the default `display` on Opus 5, and on Opus 4.6?
10. Is `adaptive` a valid `effort` value?
11. Which current model does not support `effort` at all?
12. What is the minimum `task_budget.total`, and is the budget enforced?
13. How many assistant turns are in a user question, a tool call, a tool result and a final answer?
14. What does the API do if you toggle thinking mid-turn?
15. Which two block types must be echoed back unmodified?
16. What is the one documented exception to "complete and unmodified"?
17. Which models keep only the **last** turn's thinking?
18. What must you do to thinking blocks when switching models mid-conversation?
19. Name three changes that invalidate a cached prefix.
20. Above roughly how many thinking tokens per request does the documentation recommend the Batch API?

18.37 Answers and Explanations

1. **Adaptive**: `{"type": "adaptive"}`. **Extended (legacy)**: `{"type": "enabled", "budget_tokens": N}`. Answering "enabled and disabled" confuses a mode with a state — `disabled` is a third `type` value, not a mode.
2. **Fable 5, Mythos 5, Mythos Preview, Opus 5, Sonnet 5.** Opus 4.8 and 4.7 support adaptive but default to off.

3. **A 400.** Extended thinking is legacy and is rejected on Claude 4.7 and later. On the 4.6 generation the same body still succeeds.
4. **Opus 5.** `disabled` is accepted at `high` or below and returns 400 at `xhigh` and `max`.
5. **Fable 5, Mythos 5 and Mythos Preview.** Sonnet 5 accepts it; Opus 5 accepts it conditionally.
6. **An encrypted copy of the full reasoning**, used to verify the block was generated by Claude. Opaque, cross-platform, longer on Claude 4 and later.
7. **No.** It is a summary, produced by a **different model**. No `display` setting returns the raw chain of thought.
8. **Latency — specifically time-to-first-text-token when streaming.** Not cost: you are billed for the full thinking tokens either way.
9. `omitted` **on Opus 5;** `summarized` **on Opus 4.6.** The default flipped with the newer generation.
10. **No.** `adaptive` is a thinking mode. Passing it as an effort is called out in the documentation as a mistake.
11. **Haiku 4.5.** Not "defaults to high" — unsupported.
12. **20,000 tokens**, and it is **advisory, not enforced**. The countdown is visible only to the model, and an undersized budget produces refusal-like behaviour.
13. **One.** A tool-use loop is a single assistant turn across two API calls.
14. **Nothing visible.** It silently disables thinking for that request and may strip blocks that would create an invalid structure. Detect it by checking for thinking blocks in the response.
15. `thinking` **and** `redacted_thinking`**.**
16. **Text placed in the empty** `thinking` **field of an** `omitted` **block is ignored** rather than rejected.
17. **Earlier Opus and Sonnet models, and all Haiku through Haiku 4.5.** Opus 4.5 and later Opus, and Sonnet 4.6 and later Sonnet, keep all.
18. **Strip** `thinking` **and** `redacted_thinking` **from prior assistant turns.** Other models ignore them silently but still bill them as input.
19. Any three of: switching between `adaptive`, `enabled` and `disabled`; changing `budget_tokens`; changing the resolved `effort` level; changing `display`. Setting a parameter to its default is *not* a change.
20. **About 32,000.** Such requests run long enough to hit system timeouts and open-connection limits.

18.38 Ten Practical Exercises

1. Add a fifteenth preflight rule for the 300K batch output header, rejecting it on Fable 5 and Haiku 4.5.
2. Extend `Agentic` to model interleaved thinking, where thinking occurs between every tool call rather than once per call.
3. Write a `describe(model)` function that prints the matrix row for one model and raises on the first UNKNOWN it is asked for.
4. Add a `policy_diff(old, new)` that reports which changes between two thinking policies invalidate the cache.
5. Simulate a refusal fallback at turn k and compute the wasted tokens as a function of k.

6. Add a detector that, given a response, returns whether thinking was active, and test it against a response with no thinking blocks.
7. Compute the break-even session length at which enabling thinking-block clearing saves money net of lost cache hits.
8. Extend `round_trip_ok` to report *which* block index first differs, and prove it on a reordered pair.
9. Model the one-hour cache: at what session duration does the 2× write beat the 1.25× write plus a re-write?
10. Write the CI check that fails the build if any file other than the policy module contains the string `"thinking"` as a request key.

18.39 Five Challenge Labs

1. **A thinking policy service.** One object owning mode, display, effort, budget and cache key, with preflight, degradation detection and a metrics surface. Ship it behind the interface in Figure 18.1.
2. **A migration linter.** Given a repository, find every Messages request shape and report which would 400 on a target model, with the documented reason for each.
3. **A context-clearing scheduler.** Decide per turn whether to clear old thinking blocks, trading context space against cache hits, and evaluate it against a fixed policy over simulated sessions.
4. **A cache-churn detector.** Instrument a running service to emit its cache key, and alert when the distinct-key rate per session exceeds a threshold.
5. **A redaction fuzzer.** Randomly inject `redacted_thinking` blocks into recorded conversations at a configurable rate and assert your round-trip code survives all of them.

18.40 Ten Interview Questions

1. Why is extended thinking called legacy when it is still the only mode on Haiku 4.5?
2. A colleague says `display: "omitted"` saves money. Correct them precisely.
3. Explain why a tool-use loop is one assistant turn, and give one consequence.
4. Your service gets intermittent 400s about modified thinking blocks on 20 per cent of traffic. Diagnose it.
5. Why does the preflight raise rather than return `False` for an unverified field?
6. An engineer proposes per-request effort tuning. What do you ask them to measure first?
7. Where does a long agentic conversation's input bill actually go, and why is that surprising?
8. What is the only reliable way to know whether thinking was active on a request?
9. When is `task_budget` the right tool, and when is `max_tokens`?
10. A fallback switches model mid-conversation. What breaks, and what merely costs?

18.41 Summary

Thinking on the current models is not a feature you switch on; it is a configuration with per-model rules, a round-trip protocol, a context cost and a cache identity. There are two modes and only two: **adaptive**,

which lets the model decide whether and how deeply to reason on each request, and **extended**, the legacy form in which you name a `budget_tokens` figure. Adaptive is the present tense — it is on by default on Fable 5, Mythos 5, Opus 5 and Sonnet 5, opt-in on the 4.8 through 4.6 generation, and rejected on the 4.5 generation, where extended thinking remains the only mode. Sending `{"type": "enabled"}` to Opus 5 is a 400, which makes it the loudest and easiest of this chapter's four failures.

The rest are quiet. A thinking block carries a **summary** of the reasoning, written by a different model, alongside a `signature` that is an encrypted copy of the real thing; you are billed for the full thinking tokens, not for the summary you can read, so any reconciliation based on visible text will conclude, wrongly, that the meter is broken. `display: "omitted"` — the default on current models — suppresses the summary and, when streaming, skips `thinking_delta` events entirely. It reduces latency and not cost, and the distinction matters because teams reach for it as a savings measure. Alongside `thinking` sits `effort`, which is a different parameter with a different job: it scales the whole response, text and tool calls and reasoning together, defaults to `high`, is unsupported on Haiku 4.5, and never takes the value `adaptive`.

The protocol is where silence starts. A tool-use loop is **one assistant turn**, so thinking cannot be toggled inside it — and if you try, the API does not error; it silently disables thinking for that request, leaving the absence of thinking blocks as the only detector. Within the latest assistant message, thinking blocks must be echoed back complete, in order and unmodified, and that obligation covers `redacted_thinking` blocks as well as regular ones. The filter `b["type"] == "thinking"` drops them, and on a twelve-turn conversation with one turn in four redacted it round-trips 9 of 12 where the correct filter round-trips 12. The bug is a function of the model's output rather than of your code path, which is why it reproduces on a quarter of traffic and on none of your fixtures.

Then the money. Whether prior turns' thinking stays in context is a **per-model default**: Opus 4.5 and later Opus, Sonnet 4.6 and later Sonnet, Fable 5 and Mythos 5 keep all of it; earlier models and every Haiku keep only the last turn. Keeping it is documented as a benefit — better cache hits during tool use, no performance cost — and it is; but on a twenty-turn agentic session it means **76.6 per cent of every input token billed is reasoning from a turn that already finished**, 1,489,200 tokens against 349,200, and a 1M context window that fills after 266 turns instead of 1,313. That is Chapter 16's *statelessness is quadratic* with a term nobody added to the list: the list you resend contains the model's own working.

Finally, the cache. The thinking configuration and the resolved effort level are rendered into the prompt, so they are part of the prefix: change the mode, the display, the budget or the effort and you have started a new cache. A five-level per-request effort tuner — every individual decision defensible — costs **133.3 per cent more** than one fixed configuration over twenty requests on a 40,000-token prefix. Four failures, one shape: an API surface where the loud mistake is cheap and the quiet ones are not. Fourteen documented rejections and a context accountant, both running offline in under a second, are what turn the quiet ones back into loud ones.

KEY TAKEAWAYS

1. **Two modes only.** Adaptive is current; extended (`budget_tokens`) is legacy and 400s on Claude 4.7 and later, while remaining the only mode on the 4.5 generation.
2. **Thinking is on by default on the 5-series**, so your first question is how to see it, not how to enable it.
3. **Disable rules are per-model and conditional.** Sonnet 5 yes, Fable 5 never, Opus 5 only at `high` effort or below — and unverified elsewhere, which must raise.
4. **You are billed for the full thinking tokens**, never for the summary you can see, which a different model wrote.
5. `display: "omitted"` **buys latency, not cost.**
6. `effort` **scales the whole response;** `max_tokens` **is the only hard ceiling;** `task_budget` **is advisory.** `adaptive` is never an effort.
7. **A tool-use loop is one assistant turn**, and toggling thinking inside it silently disables it rather than erroring.
8. **Echo** `thinking` **and** `redacted_thinking` **together, unmodified.** One omitted type breaks a quarter of traffic and no fixtures.
9. **Carried thinking is the dominant input cost of a long agentic session** — 76.6 per cent at twenty turns — and shortens the session that fits in a 1M window by 4.9×.
10. **The thinking configuration is part of the cache prefix.** Tune effort coarsely, or not at all.

18.42 Glossary

Term	Definition
Adaptive thinking	`{"type": "adaptive"}`. The model decides whether and how deeply to think per request.
Extended thinking	`{"type": "enabled", "budget_tokens": N}`. Legacy; the only mode on the 4.5 generation.
`thinking` block	A content block carrying a summary of Claude's reasoning plus a `signature`.
`signature`	An opaque, encrypted copy of the full reasoning, used to verify a returned block.
`redacted_thinking` block	A distinct block type with an opaque `data` field, returned when reasoning is safety-redacted.
`display`	`"summarized"` or `"omitted"`; controls whether the summary text is returned. Latency, not cost.
Summarised thinking	The readable text in a thinking block, produced by a **different model** from the one you targeted.
`effort`	`output_config.effort`; `low`–`max`, default `high`. Scales the whole response. Never `adaptive`.
`task_budget`	Beta, advisory whole-task token signal. Minimum `total` 20,000. Not on Sonnet 5.
Assistant turn	Everything from a user message to Claude's final text, including any tool-use loop.
Interleaved thinking	Thinking between tool calls; automatic under adaptive, unsupported on Haiku 4.5.

Term	Definition
Preservation	Whether prior turns' thinking stays in context. Per-model: keep-all or keep-last-turn.
`clear_thinking_20251015`	The context-editing strategy that overrides the preservation default.
Graceful degradation	The API silently disabling thinking on a mid-turn conflict, with no error.
`model_context_window_exceeded`	The stop reason returned when generation fills the window on 4.5 and later.
Cache prefix identity	The fact that thinking configuration and resolved effort are rendered into the prompt.
`reasoning_extraction`	A refusal category returned when a request tries to elicit internal reasoning as response text.

18.43 Further Reading

Primary sources, at `platform.claude.com/docs/en/build-with-claude/`, verified 30 July 2026:

- `thinking` — modes, display, the round trip, preservation, caching, the context window, encryption, redaction, limits.
- `thinking-steering-and-cost` — per-level effort behaviour, prompt-based steering, pricing.
- `thinking-tool-workflows` — the complete two-turn tool-use round trip in every SDK, and what interleaving changes.
- `thinking-troubleshooting` — the per-model configuration table, and each 400 message matched to its fix.
- `extended-thinking` — the legacy mode, budget rules, and interleaving in manual mode.
- `effort`, `task-budgets`, `context-editing`, `prompt-caching` — the four parameters this chapter touches from the outside.

Where to go next in this book. Chapter 19 puts rate limits, pricing and quotas around everything in Part 3. **Chapter 29** handles the errors this chapter's preflight is designed to prevent you from ever seeing. **Chapter 30** develops prompt caching properly, including the 1-hour duration this chapter recommends without pricing. **Part 6** builds the tool loop that section 18.7 describes as one turn. **Chapter 52** treats the context budget as a first-class constraint, where the 76.6 per cent figure from Panel F becomes a design input rather than a surprise. **Chapter 54** covers compaction and memory, which is where thinking-block clearing properly belongs.